

API

DAY 90
07/10/2025

What is An API ?

API stands for Application Programming Interface.

It's a Bridge that allows two Software Systems (or components) to Communicate and Exchange Data With Each other

In Simple Words

An API defines how one program can talk to another

Why API is Required ?

Without API's, Every System would have to :

- Understand exactly how another system is built internally
- know how to access its data, variables, or databases directly
- Be tightly coupled - so a small change in one breaks everything else

That's impossible to maintain when many applications or services need to work together

So, API solves this by acting as a middle layer

- Hides complexity
- Exposes only necessary functions
- Enables controlled, secure communication

Problems that API solve ?

Say you are building a weather app. you want to show live weather, but you don't have weather sensors or satellites.

Instead of building your own, you use:

you don't need to know how weather data was collected

```
GET https://api.openweathermap.org/data/2.5/weather?q=London
```

The Weather API responds:

```
json
{
  "temp": 22.3,
  "humidity": 60,
  "description": "clear sky"
}
```

The Problem APIs Solve

Problem (Without API)	Solution (With API)
Systems can't talk easily	APIs define a common communication format
Tight coupling between programs	APIs provide loose coupling
Security risk (direct access to database)	APIs expose only required endpoints
Rewriting code for each app	APIs let you reuse functionality
Integration complexity	APIs standardize integration

Analogy
Imagine a Restaurant

Role	Equivalent in Software
Customer	Client
Waiter	API
Kitchen	Server / Backend
Menu	API Documentation
Dish	Response

The Customer's don't go into the kitchen, Customer asks the Waiter (API) for food using the Menu (API docs). The Waiter takes your order to the kitchen (Server), and then brings the Prepared dish (response)

Customer never see how the food was cooked (implementation details are hidden), The Customer get what he asked for - neatly packaged

Advantages of APIs

Advantage	Description
Abstraction	Hides internal complexity — you just call the API
Reusability	Write once, use anywhere (many clients can use same API)
Interoperability	Connect apps written in different languages/platforms
Scalability	Frontend and backend can scale independently
Security	Only controlled access via endpoints
Faster development	Reuse third-party APIs instead of reinventing the wheel

Real-Life Examples

API	What It Does
Google Maps API	Embed maps & navigation into app
Stripe API	Process payments securely
OpenAI API	Generate AI text & images
Twitter API	Fetch tweets & post updates
Weather API	Get live weather data

Definition	Interface for communication between software components
Why Needed	To standardize, secure, and simplify system communication
Problem Solved	Integration, complexity, and coupling between systems
Analogy	Waiter between customer (client) and kitchen (server)
Advantages	Simplicity, security, reusability, scalability

Different kinds of API's exists depending on how they are accessed, where they are used and what protocol they use

🔧 1. Based on Accessibility (Who can use it?)

Type	Description	Example
1 Open APIs (Public APIs)	Available to any developer. Usually provided by companies to let others integrate their services.	Google Maps API, OpenWeather API, Twitter API
2 Partner APIs	Shared only with specific business partners or organizations. Access requires authentication and agreements.	Uber partner APIs for payment providers
3 Internal APIs (Private APIs)	Used only inside a company to connect internal systems or services. Not exposed to public.	An internal HR system connecting payroll and attendance services
4 Composite APIs	Combine multiple APIs into a single call. Useful in microservices architecture.	One API call fetches user details, order history, and recommendations together

🌐 2. Based on Communication Style or Protocol

Type	Description	Example
1 REST API (Representational State Transfer)	Most common web API type. Uses HTTP methods (GET, POST, PUT, DELETE). Lightweight and stateless.	GET /users/10 → returns user with ID 10
2 SOAP API (Simple Object Access Protocol)	Older, XML-based standard. More secure and formal, used in enterprise or banking systems.	<soap:Envelope>...</soap:Envelope>
3 GraphQL API	Developed by Facebook. Clients can specify exactly what data they need — no over-fetching.	{ user(id: 10) { name, email } }
4 gRPC API	Uses Protocol Buffers and works over HTTP/2. Very fast and efficient — used for internal microservices.	Used by Google Cloud, Netflix microservices
5 WebSocket APIs	Used for real-time, bidirectional communication.	Chat apps, live stock updates, multiplayer games

⚙️ 3. Based on Usage or Environment

Type	Description	Example
1 Web APIs	APIs accessible over the Internet using HTTP/HTTPS.	REST, GraphQL, SOAP
2 Library or Framework APIs	Expose functionalities inside programming libraries.	Java's java.util.*, Python's math module
3 OS-Level APIs	Allow apps to interact with the operating system.	Windows API, Android API, macOS Cocoa API
4 Hardware APIs	Allow software to control hardware components.	Camera API, Bluetooth API
5 Database APIs	Let apps interact with databases.	JDBC (Java Database Connectivity), MongoDB API

📁 4. Based on Data Format

Type	Data Format	Example
JSON APIs	Send and receive data in JSON (most common today).	{ "name": "Veeresh", "age": 22 }
XML APIs	Used mainly in SOAP or old systems.	<name>Veeresh</name>
Binary APIs	Use binary data (like gRPC).	Compact and fast for internal systems.

🧠 Analogy — Real-Life Example

Think of a city transportation system 🚗 🚆 🚇

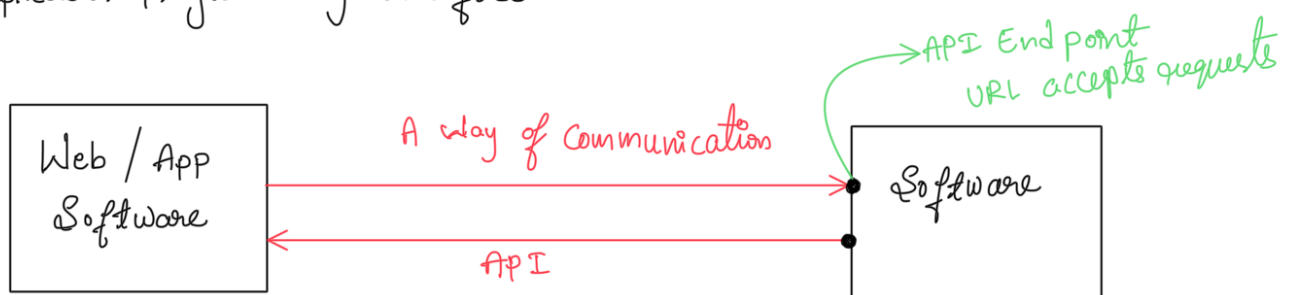
Each API type is like a different transport service:

API Type	Analogy
REST API	Public bus — simple, open, widely used
SOAP API	Government train — formal, strict schedule, secure
GraphQL	Taxi — you choose exactly where to go (customized data)
gRPC	Bullet train — fast, efficient for long distance (internal systems)
Composite API	Metro interchange — connects multiple lines (services)

💡 Summary Table

Category	Example Types	Used In
By Access	Public, Private, Partner, Composite	Business & internal systems
By Protocol	REST, SOAP, GraphQL, gRPC	Web and microservices
By Environment	Web, Library, OS, Hardware, Database	All platforms
By Data Format	JSON, XML, Binary	Data transmission

Application Programming Interface



Through API The Communication is done Between the client & Server

In the software Industry We want things to be Reused
Reusability of code solves problems faced By Many.

Example

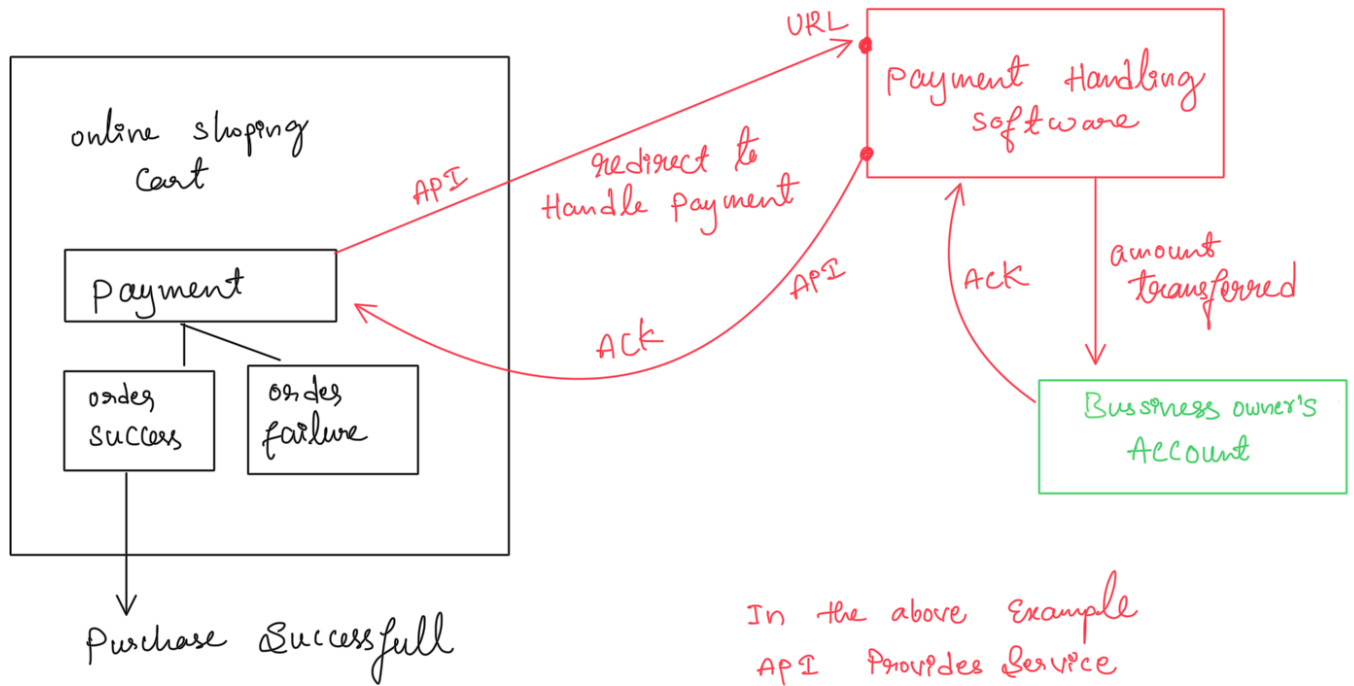
I have Built a online Shopping website → if user check's out the items (purchase)

I have to Make Sure the payment is done Securely and Safely Without any loss for anyone.

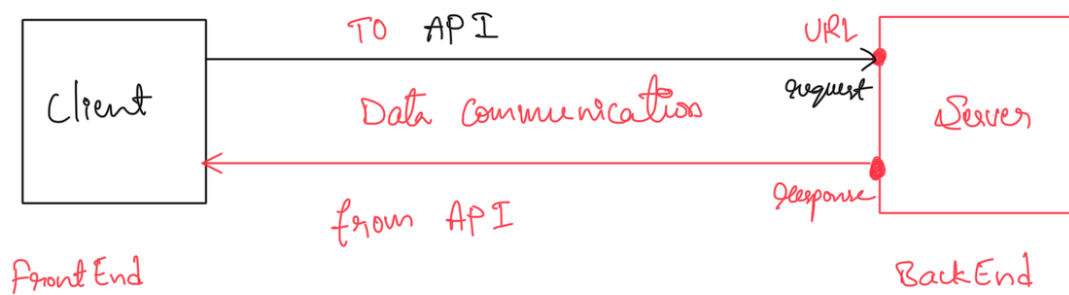
So Instead of Building Secure Payment System from the scratch,

I use already 'Built Softwares API's for Handling my website's Payment's

The API Which handles all the payments and delivers it into my account and send's acknowledgement to the client/customer as response.

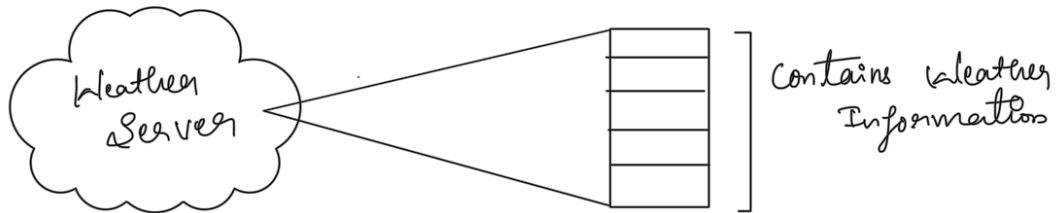


Data can be Exchanged or sent or Received or updated through API using URL (performing operations like CRUD) Between client and server.

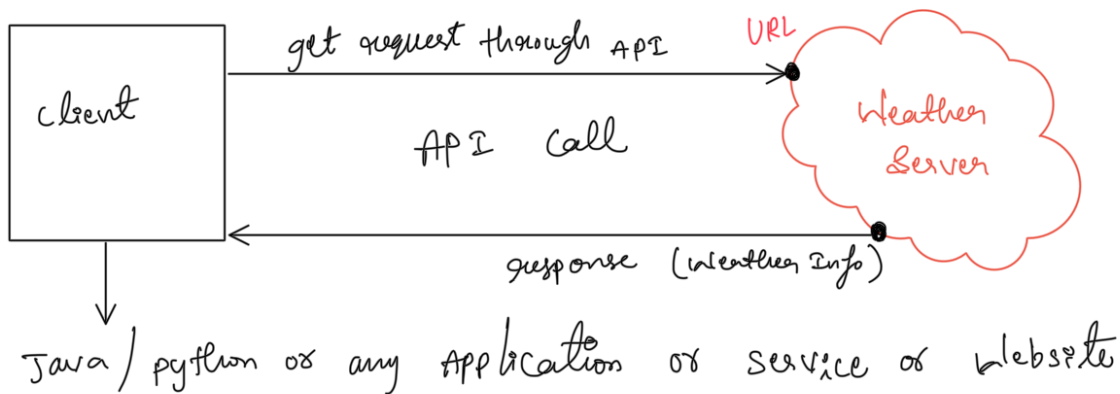


When two Software (client & server) wants to talk to each other client wants some particular information, it uses API for communication

For Example the Weather Department Built a weather API, and allows public to access the weather information of any city of India.



So the Weather Server gives an URL for Example
<https://weather.india.get/city/Date>



Code

java

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import org.json.JSONObject;

public class WeatherAPIExample {
    public static void main(String[] args) {
        // Replace with your OpenWeatherMap API key
        String apiKey = "YOUR_API_KEY";
        String city = "Bengaluru";
        String urlString = "https://api.openweathermap.org/data/2.5/weather?q="
            + city + "&appid=" + apiKey + "&units=metric";

        try {
            // Step 1: Create URL object
            URL url = new URL(urlString);

            // Step 2: Open connection
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");

            // Step 3: Read response
            BufferedReader reader = new BufferedReader(
                new InputStreamReader(conn.getInputStream()));
            StringBuilder response = new StringBuilder();
            String line;

            while ((line = reader.readLine()) != null) {
                response.append(line);
            }
            reader.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



```

// Step 4: Parse JSON response
JSONObject obj = new JSONObject(response.toString());
JSONObject main = obj.getJSONObject("main");
double temp = main.getDouble("temp");
int humidity = main.getInt("humidity");

String weatherDescription = obj.getJSONArray("weather")
    .getJSONObject(0)
    .getString("description");

// Step 5: Print results
System.out.println("☀️ Weather in " + city + ":");
System.out.println("Temperature: " + temp + "°C");
System.out.println("Humidity: " + humidity + "%");
System.out.println("Description: " + weatherDescription);

} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
}
}
}

```

Then compile and run using:

bash

```

javac -cp .:json-20231013.jar WeatherAPIExample.java
java -cp .:json-20231013.jar WeatherAPIExample

```

*Include json
libraries manually*

(Use ; instead of : on Windows)

How It Works Internally

1. You build a **URL** with query parameters (city, apiKey).
2. Java's `URLConnection` sends a **GET request** to the server.
3. The Weather API (server) sends back a **JSON response**.
4. You parse that JSON to extract fields like temp, humidity, etc.
5. Finally, you display the data.